



AI Documentation

ABDULRAHMAN TAMER

231004709

ABDIAZIIZ MAHAD MOHAMED

231015214

AHMED ELGAZAIKY 231006741

SEIF ELDEEN EMAD 231006313

ZIAD BELAL 231014487

How Random Forest (RF) and Neural Network (NN) Work

Overview

This project uses **two different machine learning algorithms** to predict football player performance and market value:

1. **Random Forest (RF)** - Ensemble tree-based method
2. **Neural Network (NN)** - Multi-layer perceptron with backpropagation

Both models take the same input: **4 features** (goals, assists, minutes played, age) and predict performance scores and market values.

Random Forest (RF) - How It Works

Concept

Random Forest is an **ensemble method** that combines multiple decision trees. Think of it as asking many experts (trees) and taking their average opinion.

Architecture

Input Features (4 values) → Decision Tree 1 → Prediction 1

Decision Tree 2 → Prediction 2

Decision Tree 3 → Prediction 3

...

Decision Tree 100 → Prediction

100

Average of all predictions → Final Prediction

Step-by-Step

1. **Training Phase** (model_trainer.py lines 195-211)
 - Creates 100 decision trees
 - Each tree sees a random subset of training data (bootstrap sampling)
 - Each tree uses a random subset of features at each split
 - Each tree learns different patterns and votes
 - The final prediction is the average of all tree predictions
2. **Decision Tree Structure**

3. **❓ If Goals < 10**
4. Low Value
5. **❓ If Goals ≥ 10**
6. **If Age < 25**
7. **If Assists > 5**
8. High Value
9. **If Assists ≤ 5**
10. Medium Value
11. **If Age ≥ 25**
12. **Prediction Phase** (predictor.py lines 106-109)
 - Input features are scaled
 - Each of the 100 trees makes a prediction
 - Final prediction = Average of all tree predictions

Advantages of Random Forest

- Robust – Less prone to overfitting
 - Fast training – Parallel tree building
 - Feature importance – Can see which features matter most
 - Handles non-linear relationships
 - No need for feature scaling (though scaling is used for consistency)
-

Neural Network (NN) - How It Works

Concept

Neural Network mimics the human brain using **neurons connected in layers**. It learns complex patterns through **forward propagation** and **backpropagation**.

Architecture

- Input Layer: 4 neurons
- Hidden Layer 1: 64 neurons (ReLU activation)

- Hidden Layer 2: 32 neurons (ReLU activation)
- Hidden Layer 3: 16 neurons (ReLU activation)
- Output Layer: 1 neuron (linear activation)

Step-by-Step

1. **Initialization** (neural_network.py lines 22-50)
 - Weight matrices and bias vectors are created for each layer
 - He initialization is used for weights
 - Biases start at 0
2. **Forward Propagation** (neural_network.py lines 66-90)
 - Computes output layer by layer using:

$$z = (\text{previous_layer_output} \times \text{weights}) + \text{biases}$$

$$a = \text{activation_function}(z)$$
 - ReLU activation: $\text{output} = \max(0, \text{input})$
3. **Training Phase** (neural_network.py lines 102-180)
 - Uses gradient descent and backpropagation
 - Forward pass → Calculate error → Backward pass → Update weights
 - Iterates for multiple epochs
4. **Prediction Phase** (predictor.py lines 106-107)
 - Scale input features
 - Forward pass through trained network
 - Output: single prediction value

Advantages of Neural Network

- Learns complex patterns
- Flexible architecture
- Good for large datasets
- Can approximate any function
- Requires more data

- Slower training than RF
- Harder to interpret (black box)

Comparison: Random Forest vs Neural Network

Training Process

Aspect	Random Forest	Neural Network
Method	Builds 100 independent trees	Trains interconnected layers
Learning	One-shot (builds trees once)	Iterative (100 epochs)
Speed	Fast (parallel)	Slower (sequential)
Data Needed	Smaller datasets	Needs more data
Interpretability	Feature importance visible	Black box

Prediction Process

Aspect	Random Forest	Neural Network
Process	Each tree votes, average	Forward pass through layers
Speed	Very fast	Fast (single forward pass)
Complexity	Simple averaging	Matrix multiplications

How They Work Together in This Project

- System automatically chooses which model to use:
 1. Try Neural Network first
 2. If not found, try Random Forest
 3. If neither, use fallback calculations
- Both models are trained independently on the same data

Real Example

- User searches for “Messi”
 - Extract player data (Goals=30, Assists=15, Minutes=2800, Age=35)
 - Features are scaled and fed into both models
 - Predictions:
 - Performance score ≈ 0.90
 - Predicted goals ≈ 0.7
 - Predicted assists ≈ 0.4
 - Market value $\approx \$45.2\text{M}$
-

Key Code Locations

- **Random Forest**
 - Training: model_trainer.py lines 195-211
 - Loading: model_trainer.py lines 243-262
 - Prediction: predictor.py lines 106-109
 - **Neural Network**
 - Class Definition: neural_network.py lines 12-326
 - Training: neural_network.py lines 102-180
 - Loading: predictor.py lines 45-61
 - Prediction: predictor.py lines 106-107
-

Math Overview

Random Forest

Final Prediction = Average of all tree predictions

Neural Network

- Each layer: $z = X \times W + b$, $a = \text{ReLU}(z)$
- Loss = MSE = $(1/m) \times \sum (\text{prediction} - \text{actual})^2$

- Gradient descent: $W = W - \alpha \times \partial \text{Loss} / \partial W$